Google Form Trigger — The Full Flow

How the webhook is created, saved, triggered, and executed — end to end

Nodebase • Next.js Route Handler + Apps Script + Upstash (rate-limit & idempotency) + Inngest execution engine • Generated 2026

Google Apps Script | PUBLIC webhook, no auth | Rate limited 30 req/min | Idempotent by responseId | Inngest executeWorkflow | Realtime node status

1. Overview

A Google Form submission is turned into a workflow run in five stages. The "webhook" is not a separately stored record — it is **any workflow that contains a `GOOGLE_FORM_TRIGGER` node**. The webhook URL is derived from the workflow id at render time, so saving the node *is* publishing the webhook:

1. **Create** — add a *Google Form* trigger node in the editor; its settings dialog builds the webhook URL and hands you a ready-to-paste Apps Script.
2. **Save** — the node is persisted with the whole workflow in one Postgres transaction (`workflows.update`).
3. **Trigger** — on form submit, Apps Script POSTs a JSON payload to `/api/webhooks/google-form?workflowId=...`.
4. **Guard** — the route rate-limits per workflow id (Upstash, default 30/min) and dedupes by response id (Redis `SET NX`), then dispatches to Inngest.
5. **Execute** — `executeWorkflow` creates an Execution, topologically sorts the graph, runs each node with the form data in `context.googleForm`, publishes realtime status, and writes a SUCCESS/FAILED result.

Files involved

Concern	File
Webhook URL + setup dialog	src/features/trigger/components/google-form-trigger/dialog.tsx
Apps Script generator	src/features/trigger/components/google-form-trigger/utils.ts
Node on canvas + realtime	src/features/trigger/components/google-form-trigger/node.tsx
Realtime token (server action)	src/features/trigger/components/google-form-trigger/actions.ts
Trigger executor	src/features/trigger/components/google-form-trigger/executor.ts
Realtime channel	src/inngest/channels/google-form-trigger.ts
HTTP endpoint	src/app/api/webhooks/google-form/route.ts
Rate limiter	src/lib/rate-limit.ts
Idempotency key	src/lib/idempotency.ts
Dispatch helper	src/inngest/utils.ts
Execution engine	src/inngest/functions.ts

2. Creating the Webhook

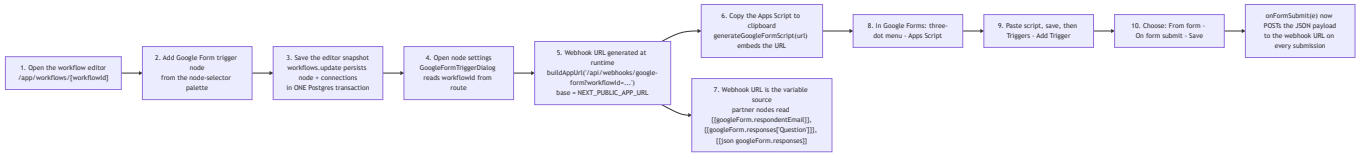


Figure 1 — From dragging a trigger node onto the canvas to a live "on form submit" trigger

Webhook URL anatomy

```
https://<NEXT_PUBLIC_APP_URL>/api/webhooks/google-form?workflowId=<workflowId>
            |            base (buildAppUrl)            |  |          route path          |  |   query: route param   |
```

- Base = `NEXT_PUBLIC_APP_URL` (falls back to `http://localhost:3000`), built by `buildAppUrl`.
- `workflowId` is the only parameter; it identifies the receiving workflow and doubles as the rate-limit identifier.
- The URL is generated at runtime in the dialog — nothing is stored; it simply reflects the workflow the node lives in.

The generated Google Apps Script (utils.ts)

```
function onFormSubmit(e) {
  var formResponse = e.response;
  var itemResponses = formResponse.getItemResponses();
  var responses = {};
  for (var i = 0; i < itemResponses.length; i++) {
    var ir = itemResponses[i];
    responses[ir.getItem().getTitle()] = ir.getResponse();
  }
  var payload = {
    formId: e.source.getId(),
    formTitle: e.source.getTitle(),
    responseId: formResponse.getId(),
    timestamp: formResponse.getTimestamp(),
    respondentEmail: formResponse.getRespondentEmail(),
    responses: responses
  };
  var options = { 'method': 'post', 'contentType': 'application/json',
                  'payload': JSON.stringify(payload) };
  var WEBHOOK_URL = 'https://.../api/webhooks/google-form?workflowId=...';
  try { UrlFetchApp.fetch(WEBHOOK_URL, options); }
  catch (error) { console.error('Webhook failed:', error); }
}
```

Variables available to downstream nodes

Expression	What it yields
`{{googleForm.respondentEmail}}`	Respondent's email (only if the form collects it)
`{{googleForm.formTitle}}`	Form title
`{{googleForm.responseId}}`	Unique submission id (also the dedupe key)
`{{googleForm.responses['Question Name']}}`	Answer to a specific question
`{{json googleForm.responses}}`	Whole answers object as a JSON string (`json` Handlebars helper)

3. How the Webhook Is Saved

There is deliberately **no `Webhook` table**. The webhook is implied by a `GOOGLE_FORM_TRIGGER` node on a workflow. What is really persisted is the workflow graph itself, so the "webhook" is durable exactly as long as the node is.

```
// workflows.update - single Postgres transaction: replace the whole graph
await prisma.$transaction(async (tx) => {
  await tx.connection.deleteMany({ where: { workflowId } });
  await tx.node.deleteMany({ where: { workflowId } });
  await tx.node.createMany({ data: nodes });   // includes type: GOOGLE_FORM_TRIGGER
  await tx.connection.createMany({ data: edges });
});
```

What the three sources of truth hold

Where	What it stores	Schema
workflow	The owner (`userId`) and name; gives the execution engine its FK root	`Workflow { id, name, userId }`
node	One row per canvas node, `type = GOOGLE_FORM_TRIGGER`, JSONB `data`/`position`	`Node { id, workflowId, type, data Json, position Json }`
connection	Edges `fromNodeId -> toNodeId` that drive the topological sort	`Connection { id, workflowId, fromNodeId, toNodeId }`

Consequences: deleting the workflow (or the node) removes the endpoint automatically (no orphaned URLs); the URL is stable for the life of the workflow because it is derived from the id, not stored.

4. How It Is Triggered

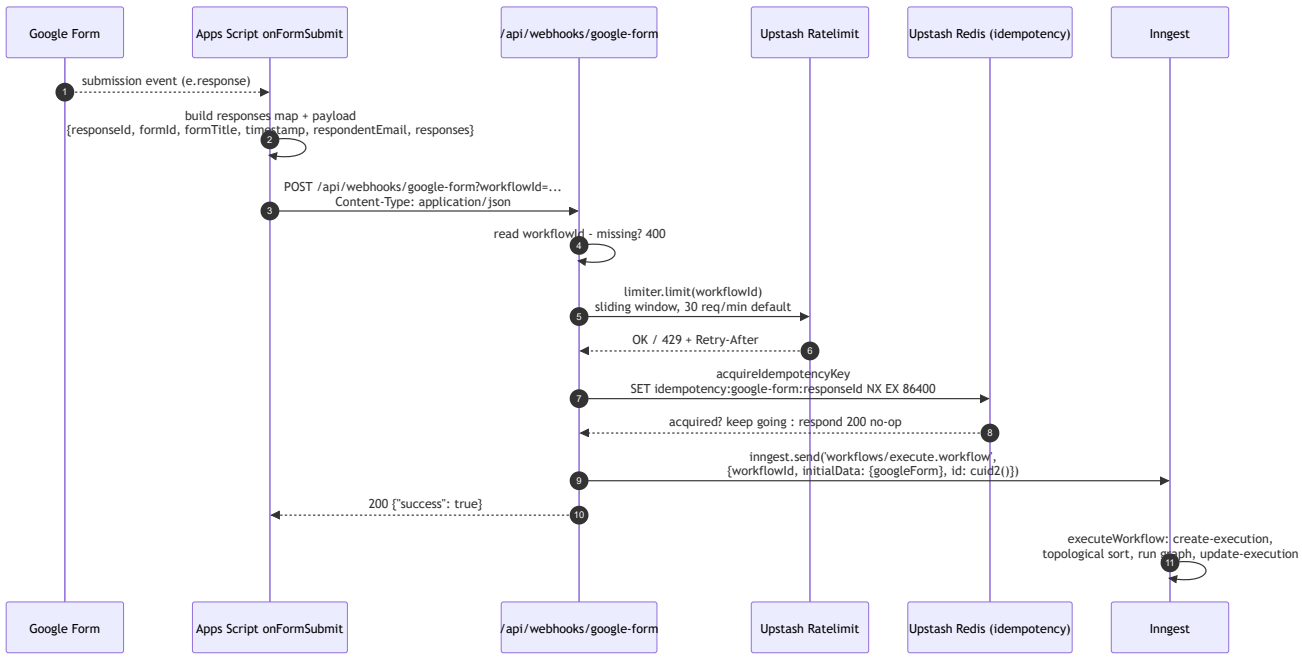


Figure 2 — A submission travels Google Form → Apps Script → Nodebase route (guards) → Inngest

When a respondent hits *Submit*, Apps Script fires `onFormSubmit(e)`. Script builds the payload and does a plain HTTPS POST. Nodebase responds quickly ("fire and forget" protocol); the heavy lifting happens inside Inngest.

Payload that Apps Script sends

```
{
  "formId": "1FAIpQLSdN9K...",
  "formTitle": "Customer Lead Form",
  "responseId": "2_ABaOnuf...",
  "timestamp": "2026-10-12T09:41:03.789Z",
  "respondentEmail": "ada@example.com",
  "responses": {
    "Full name":     "Ada Lovelace",
    "Which plan":    ["Pro", "Teams"],
    "Message":       "Need onboarding help"
  }
}
```

What the route handler does (route.ts POST)

1. **Read `workflowId`** from the query string; missing → `400`.
2. **Rate limit** — `enforceRateLimit(webhookGoogleFormLimiter, workflowId)`; sliding window 30 req/min (env `RATE_LIMIT_GOOGLE_FORM_PER_MIN`), key prefix `ratelimit:webhook:google-form`. Exceeded → `429 + Retry-After`.
3. **Parse JSON body** and normalize into `formData { formId, formTitle, responseId, timestamp, respondentEmail, responses, raw }`.
4. **Dispatch with dedupe** — `sendWorkflowExecution({ workflowId, idempotencyKey: "google-form:"+responseId, initialData: { googleForm: formData } })`.
5. **Reply `200 {"success": true}`** to Apps Script immediately.

Guard rails (and where each is enforced)

Risk	Defense	Where
Missing/misconfigured target	Missing `workflowId` → 400	route.ts

Abuse / misbehaving form double-fires	Sliding window 30 req/min per workflow id	rate-limit.ts (Upstash)
Duplicate delivery (Apps Script retries, double trigger)	`SET idempotency:google-form:<responseId> NX EX 86400` → second hit answers 200 no-op, no event sent	idempotency.ts
Malformed body	try/catch → 500	route.ts
Duplicate execution row for one run	unique `inngestEventId` on Execution	schema (DB)

5. How It Executes

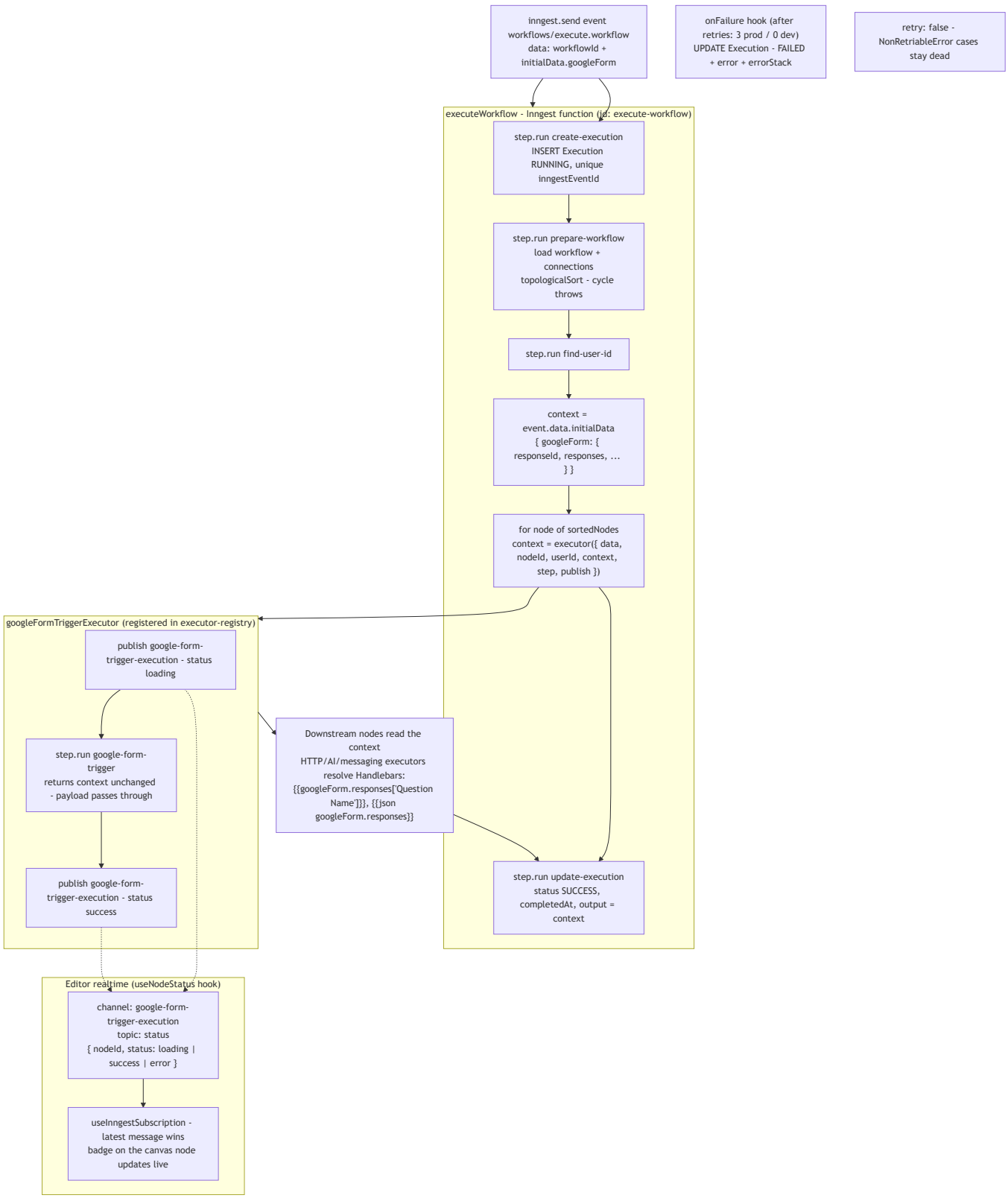


Figure 3 — executeWorkflow: execution record → graph sort → trigger executor → downstream nodes → result

Step by step

1. **Dispatch** — `inngest.send("workflows/execute.workflow", { workflowId, initialData, id })` with a cuid2 event id.

2. **create-execution** — inserts `Execution` (status `RUNNING`, `inngestEventId = event.id`). Unique constraint ties the row 1:1 to the Inngest run.
3. **prepare-workflow** — loads workflow + nodes + connections, runs `topologicalSort`; a cycle throws before any node executes.
4. **find-user-id** — resolves the workflow owner for executor context.
5. **Context seed** — `context = event.data.initialData`, i.e. `{ googleForm: { ... } }`.
6. **googleFormTriggerExecutor** — publishes `loading` → `step.run` returns context untouched (payload is already in context) → publishes `success`.
7. **Downstream nodes** — HTTP/AI/messaging executors compile Handlebars against context, e.g. `{{googleForm.responses['Question Name']}}` or `{{json googleForm.responses}}`.
8. **update-execution** — marks `SUCCESS`, sets `completedAt` and stores the final `output = context`.
9. **Failure** — retries 3 in prod / 0 in dev; after exhaustion the `onFailure` hook flips the row to `FAILED` with `error` + `errorStack`.

How an action node consumes the form data

```
// http-request executor - Handlebars compiled against the shared context
const endpoint = Handlebars.compile(data.endpoint)(context);
const body     = Handlebars.compile(data.body || "{}")(context);

// example body a user writes in the node config:
//   { "email": "{{googleForm.respondentEmail}}",
//     "plan":  "{{googleForm.responses['Which plan']}}",
//     "all":   {{json googleForm.responses}} }
```

6. Realtime Status in the Editor

While the run executes, the canvas node lights up because the trigger executor publishes to an Inngest realtime channel and the editor subscribes to it.

Concern	Detail
Channel	`google-form-trigger-execution` (channel name const `GOOGLE_FORM_TRIGGER_CHANNEL_NAME`)
Topic	`status`, typed as `{ nodeId, status: "loading" | "success" | "error" }`
Publisher	executor via the `publish` helper injected by Inngest
Subscription	`useNodeStatus` hook (React) → `useInngestSubscription`; refresh token obtained via server action `fetchGoogleFormTriggerRealtimeToken` (getSubscriptionToken)
Rendering	latest matching message wins; `BaseTriggerNode` badge shows initial / loading / success / error

Realtime playing field

The same pattern applies to every node type: each has a tiny channel file in `src/inngest/channels/`, its executor publishes loading/success/error, and the corresponding `node.tsx` hooks up the subscription. The Google Form trigger is unusual only in that its executor is a pass-through: it adds no data, because `initialData.googleForm` already seeded the context.

7. End-to-End Recap & Things to Watch

The whole timeline

1. User drops a *Google Form* node and copies the generated URL + Apps Script.
2. User pastes the script into the form's Apps Script editor and adds a *From form → On form submit* trigger.
3. Edition save persists the node/edges atomically (`workflows.update` transaction) — this **is** the webhook registration.
4. On submit, Apps Script POSTs the JSON payload to `/api/webhooks/google-form?workflowId=...`.
5. The route enforces the per-workflow rate limit, then `SET NX` idempotency key on `google-form:<responseId>` (24h TTL) to swallow retries.
6. It normalizes the payload into `initialData.googleForm` and `inngest.sends` the workload event.
7. Inngest creates the Execution, topologically sorts, runs every node sharing `context`, publishes realtime status, and finalizes SUCCESS / FAILED.
8. Executions history is stored on the workflow and rendered in the UI; the canvas badges update live via the realtime channel.

Operational notes & gotchas

- **Public endpoint, no auth** — security posture is rate limiting + idempotency, not credentials. Anyone with the URL can fire the workflow; do not put sensitive-only logic on a plain Google Form path.
- **Dedupe scope** — idempotency is per `responseId` with a 24h TTL. Re-firing after 24h, or submissions without a stable responseId, will run again.
- **Retries can repeat side effects** — Inngest retries (3 in prod) re-run the whole pipeline after a transient failure; a webhook-sending action node may fire more than once.
- **Cycle safety** — a cyclic graph fails during `prepare-workflow`, before any action node runs.
- **Rate limit default** — 30 submissions/min per workflow (upstream form bursts are the usual trigger for 429s).